



Video Analytics

(Merged - AIMLCZG531/
AIMLCZG540)

ASSIGNMENT - I

Comparative Analysis of Classical and Deep Learning Approaches for Video-Based Activity Recognition

Student ID: 2024ab05275

1. Introduction

1.1 Problem Statement and Objectives

- **Problem Statement:**
 - **Develop a comparative video classification system** that implements both classical and modern approaches to categorize videos into predefined classes.
 - The system should **demonstrate the evolution from hand-crafted features with classical ML to learned representations with deep learning**, providing insights into the strengths, limitations, and appropriate use cases for each approach.
- **Objectives**
 - **Design a unified video classification framework** supporting both classical and deep learning approaches.

- Implement hand-crafted feature-based pipelines using **classical machine learning** classifiers.
- Implement **deep learning-based models** that learn spatiotemporal representations automatically.
- Perform a comprehensive **comparison based on performance, computational cost, and model complexity**.
- **Analyze strengths, limitations, and suitable application scenarios** for each classification paradigm.

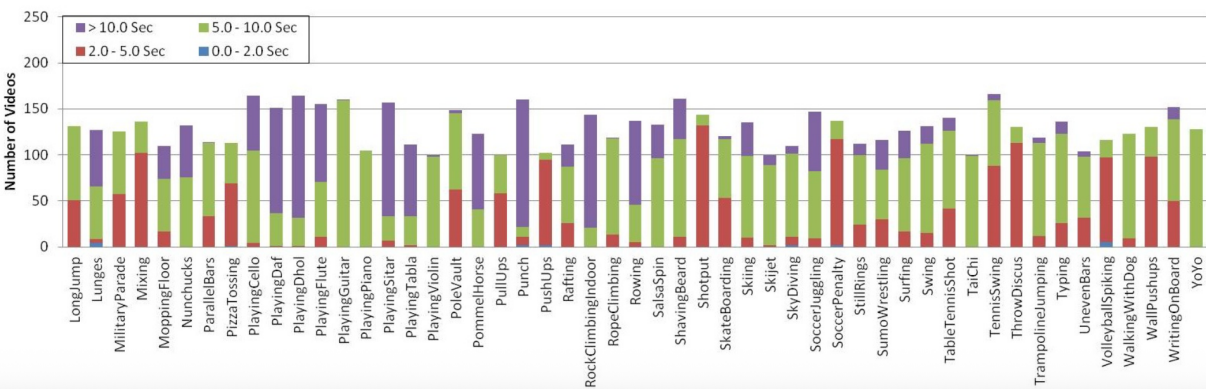
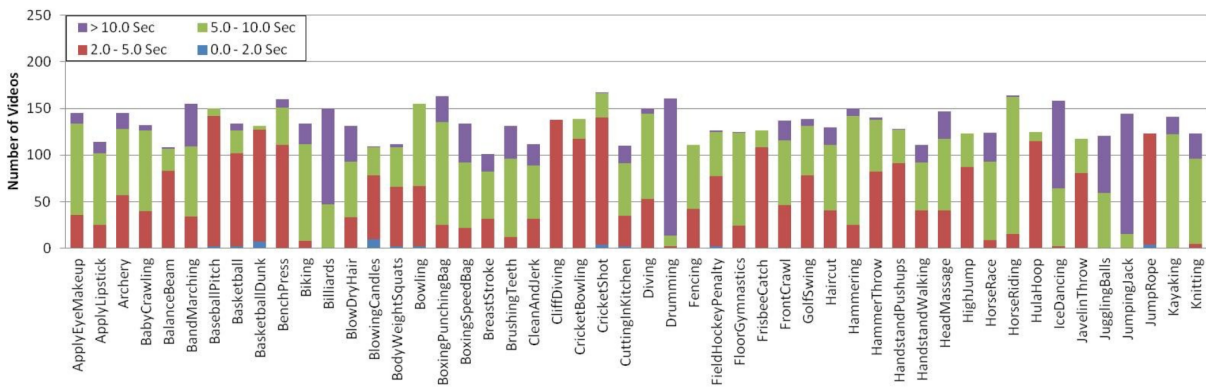
1.2 Dataset Description and Motivation

- **Data set:** <https://www.crcv.ucf.edu/data/UCF101.php>
- **Dataset Details:**
 - **Number of Classes:** **101 distinct human action categories** (sports, body motion, human-object interaction, playing instruments, etc.).
 - **Total Videos:** **13,320 realistic video** clips collected from YouTube.
 - **Video Characteristics:**
 - **Format:** **AVI**
 - **Typical resolution:** **around 320×240 pixels**
 - **Duration:** **~1–30 seconds per video clip**.
 - Videos are **grouped into 25 groups** for each action category.
- **Motivation of the UCF-101 Dataset**
 - The UCF-101 dataset was introduced **to address the need for a large-scale, realistic benchmark for human action recognition in videos**.
 - Earlier action recognition datasets were:
 - Small in size



- Captured in controlled or laboratory environments
- Limited in visual diversity and real-world complexity
- o UCF-101 was designed to capture actions “in the wild”, using **videos collected from YouTube**, reflecting:
 - Realistic camera motion
 - Background clutter
 - Occlusions
 - Illumination changes
 - Large intra-class variability
- o The dataset aims to **support research** on:
 - **Spatio-temporal understanding of human actions**
 - **Robust feature extraction from videos**
 - **Learning motion patterns under unconstrained conditions**





1.3 Overview of Approaches

1.3.1 Classical Machine Learning Approach

- Operates on handcrafted, video-level feature representations
- Raw videos are converted into fixed-length numerical feature vectors
- Feature extraction focuses on:
 - Color distribution statistics
 - Aggregated frame-level information
- Temporal information is captured **implicitly** through feature aggregation
- Features are standardized using z-score

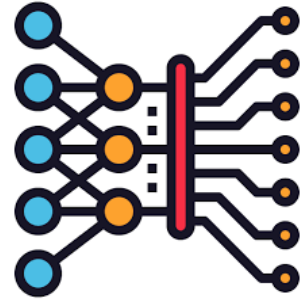


normalization (StandardScaler)

- Scaler is fitted only on training data
- Classification is performed using traditional classifiers such as:
 - Support Vector Machines (SVM)
 - k-Nearest Neighbors (k-NN)
 - Random Forests
- Emphasizes:
 - Lower computational cost
 - Model interpretability
 - Faster training on limited data

1.3.2 Deep Learning Approach:

- Operates directly on preprocessed video frame sequences
- Each video is represented as a **5D tensor**:
(frames, height, width, channels)
- Explicitly models spatio-temporal patterns in videos
- Temporal consistency is enforced using **fixed-length frame sequences**
- Frame-level inputs are normalized to the **[0, 1]** range
- Feature learning is performed **automatically** by the network
- Classification is learned end-to-end using neural network architectures
- Emphasizes:
 - Higher representation capacity
 - Ability to learn complex motion patterns
 - Reduced reliance on manual feature design

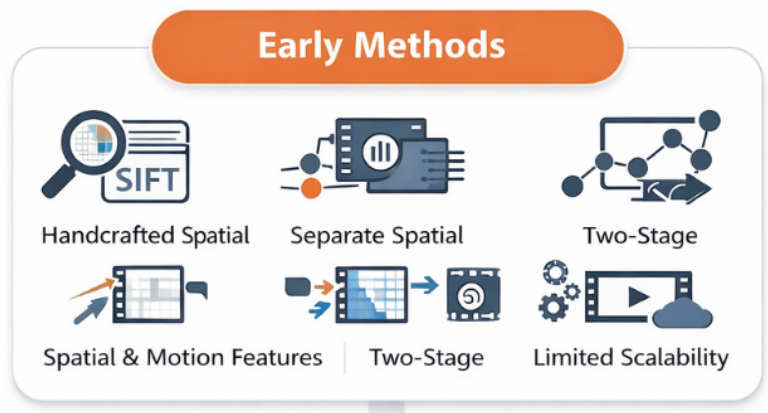


2. Background and Related Work

2.1 Evolution of Video Classification

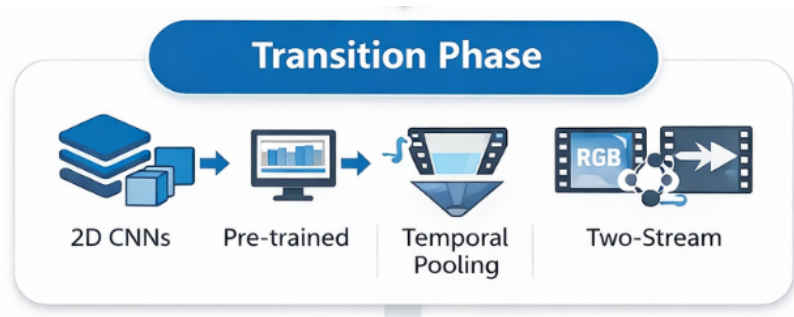
2.1.1 Early Methods

1. **Relied on handcrafted spatial features** such as SIFT, HOG, and color histograms to represent individual video frames.
2. Used **handcrafted motion features** like Optical Flow, Motion History Images (MHI), and Improved Dense Trajectories (IDT).
3. **Treated spatial and temporal information separately**, often extracting frame-level and motion features independently.
4. Followed a **two-stage pipeline**: **manual feature extraction followed by classical classifiers such as SVMs, k-NN, or Random Forests**.
5. **Limited scalability and performance** due to **lack of end-to-end learning** and **poor generalization** to complex video dynamics.



2.1.2 Transition Phase

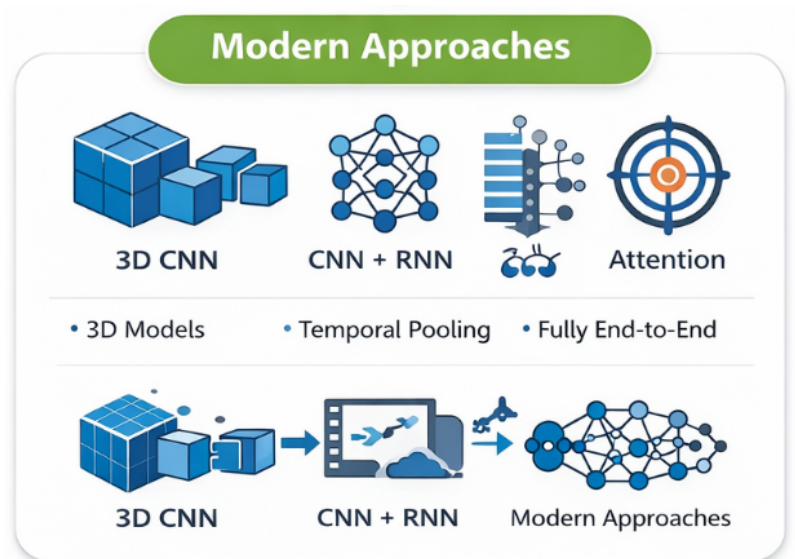
1. Introduction of **Convolutional Neural Networks (CNNs)** for automatic spatial feature learning from individual frames.



2. **Pre-trained 2D CNNs (e.g., AlexNet, VGG) were fine-tuned** for video frame classification.
3. Temporal information was handled using **post-hoc aggregation techniques** such as average pooling, max pooling, or Bag-of-Frames.
4. **Emergence of two-stream architectures**, where RGB frames and optical flow were processed by separate CNNs.
5. Partial end-to-end learning, but **temporal modeling remained shallow and indirect**.

2.1.3 Modern Approaches

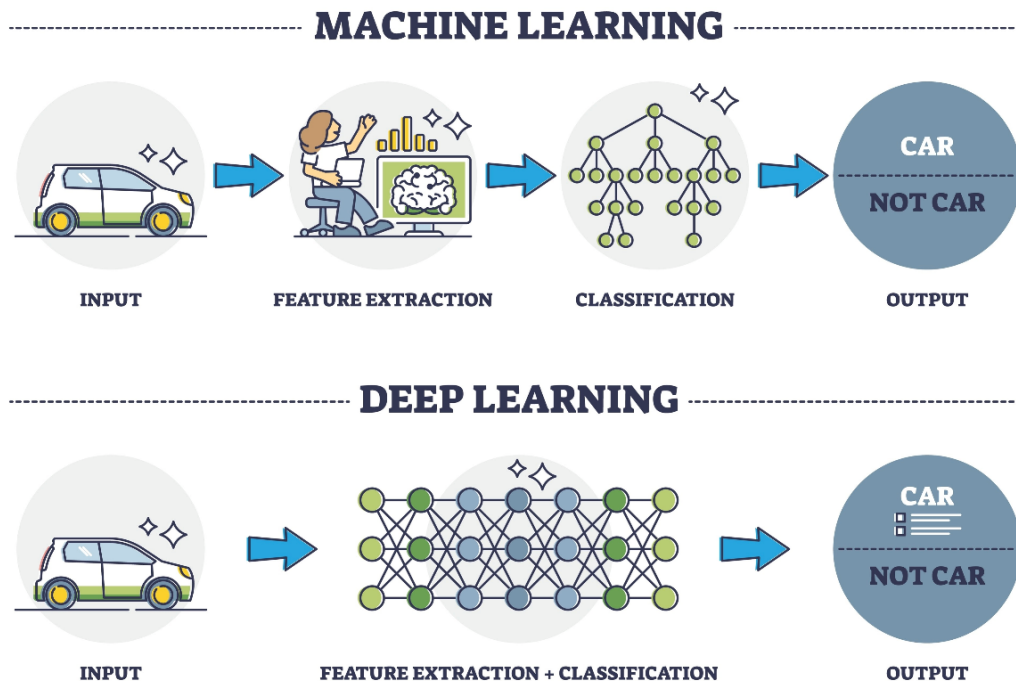
1. **Adoption of 3D Convolutional Neural Networks (3D CNNs)** to jointly learn spatial and temporal features using spatiotemporal kernels.
2. **Development of CNN-RNN hybrids**, where CNNs extract spatial features and RNNs/LSTMs/GRUs model temporal dependencies.
3. Use of **temporal pooling mechanisms** (temporal max/average pooling) to summarize long video sequences efficiently.
4. **Integration of attention mechanisms** to focus on salient frames or motion patterns critical for classification.
5. **Fully end-to-end trainable architectures** achieving state-of-the-art performance on large-scale video datasets.



2.2 Classical vs Deep Learning Paradigms

The choice between paradigms involves a fundamental trade-off between **efficiency** and **raw performance**.

- **Classical ML** is ideal for **constrained resources and smaller datasets**
- **Deep Learning** excels in **complex, large-scale tasks** where high accuracy is paramount.



Feature Learning: The shift moves **from human-engineered "handcrafted" features to deep architectures** that automatically learn hierarchical representations from raw data.

Performance Scaling: Classical ML accuracy tends to plateau early, whereas **Deep Learning performance scales significantly** as the volume of training data increases.

Data Type Specialization: While classical methods excel at structured tabular data, the deep learning **shift was driven by the need to process unstructured data** like video, images, and natural language.

Hardware Dependency: The paradigm has moved **from CPU-based processing to a heavy reliance on high-performance GPUs and TPUs** to handle parallel matrix computations.

Training Efficiency: Classical models offer **near-instant training times (e.g., k-NN at 0.07s)**, while **deep learning requires extensive temporal resources** for backpropagation.

Interpretability: The **shift has traded "white-box" transparency, where decision logic is clear, for "black-box" complexity where internal neural weights are difficult to interpret.**

Model Complexity: Modern paradigms have transitioned **from low-parameter statistical models to massive architectures containing millions of trainable parameters.**

Memory Footprint: Deployment has shifted **from lightweight, kilobyte-sized models to memory-intensive files** requiring hundreds of megabytes of storage and RAM.

Generalization and Transfer: Deep learning introduced the ability to transfer **knowledge from pre-trained models to new tasks**, a capability largely absent in traditional task-specific ML.

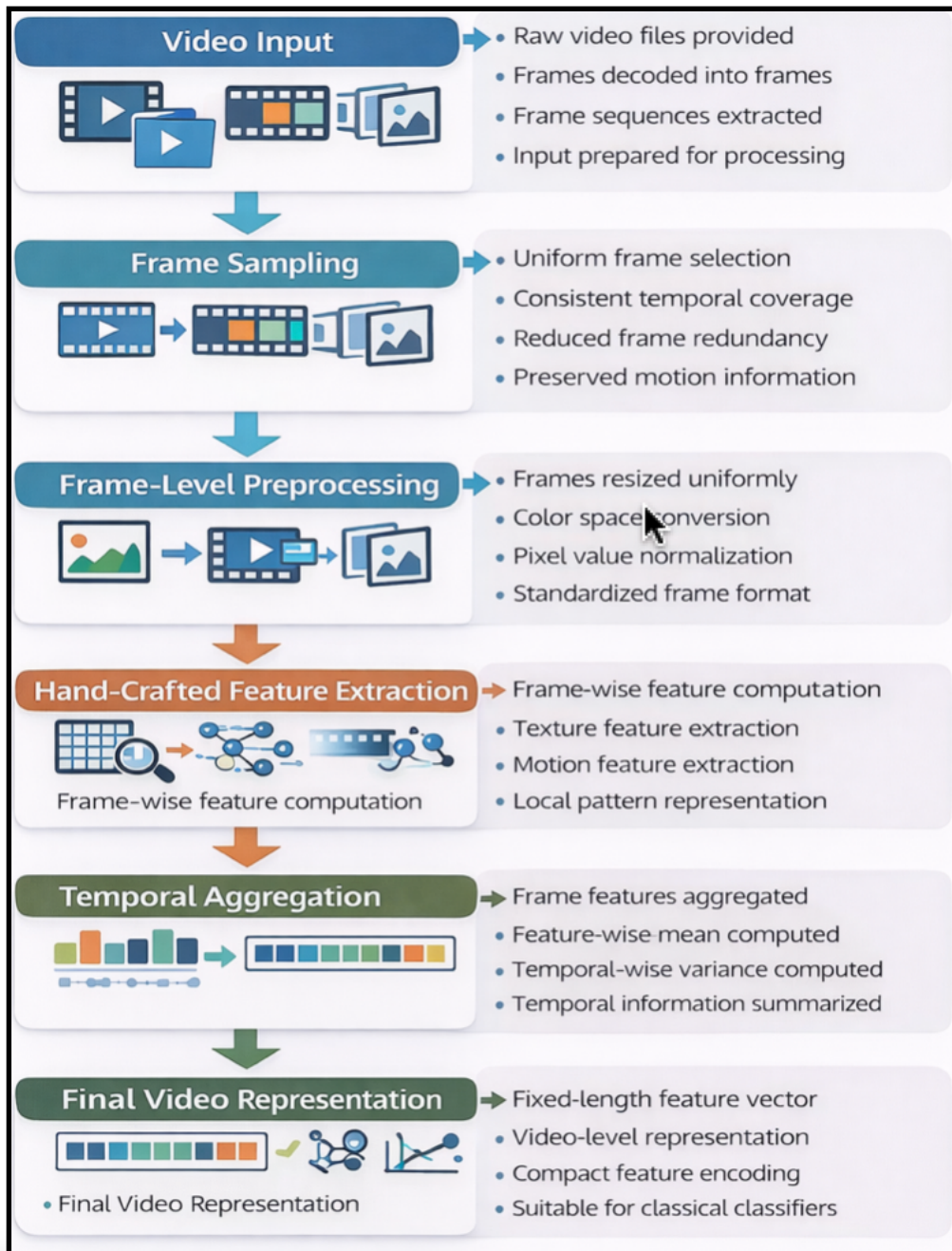
Deployment Strategy: The evolution has bifurcated the industry, **utilizing classical ML for resource-constrained edge devices and deep learning for high-precision cloud-based analytics.**

CLASSICAL ML vs. DEEP LEARNING: — A PARADIGM SHIFT —	
Classical Machine Learning	Deep Learning
 Feature Learning: Handcrafted	 Feature Learning: Automatic
 Domain Knowledge: Required	 Domain Knowledge: Not Required
 Computation: Fast	 Inference Speed: Slower
 Interpretability: High	 Generalization: Strong
 End-to-End Learning: No	 End-to-End Learning: Yes
✓ Classical ML (SVM, RF, k-NN) offers interpretability & speed, ideal for edge devices.	✓ Deep Learning (CNNs) offers superior accuracy, but at a high computational & temporal cost.
✓ Classical ML (SVM, RF, k-NN) offers interpretability & speed, ideal for edge devices. ✓ Deep Learning (CNNs) offers superior accuracy, but at a high computational & temporal cost.	

3. Methodology

3.1 Classical Machine Learning Approach

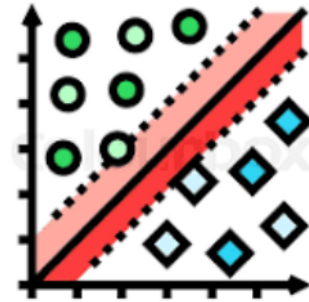
3.1.1 Feature Extraction Techniques



3.1.2 Machine Learning Algorithms

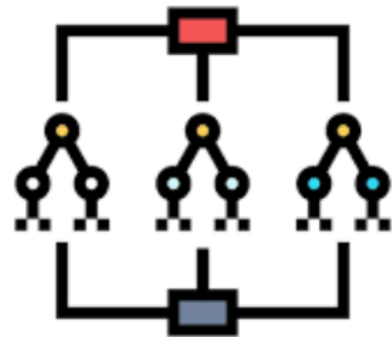
Support Vector Machine (SVM):

- **Learns a maximum-margin decision boundary**, which improves generalization.
- **Performs well in high-dimensional feature spaces**, making it suitable for handcrafted video features.
- **Supports kernel functions.**
- Less sensitive to the curse of dimensionality
- Effective when the number of features is large relative to the number of training samples.



Random Forest:

- **Ensemble of decision trees**, where each tree is trained on a random subset of data and features.
- **Reduces overfitting by averaging predictions across multiple trees.**
- Robust to noisy features and redundant information.
- Can handle non-linear class boundaries without explicit feature scaling.
- **Provides feature importance estimates.**



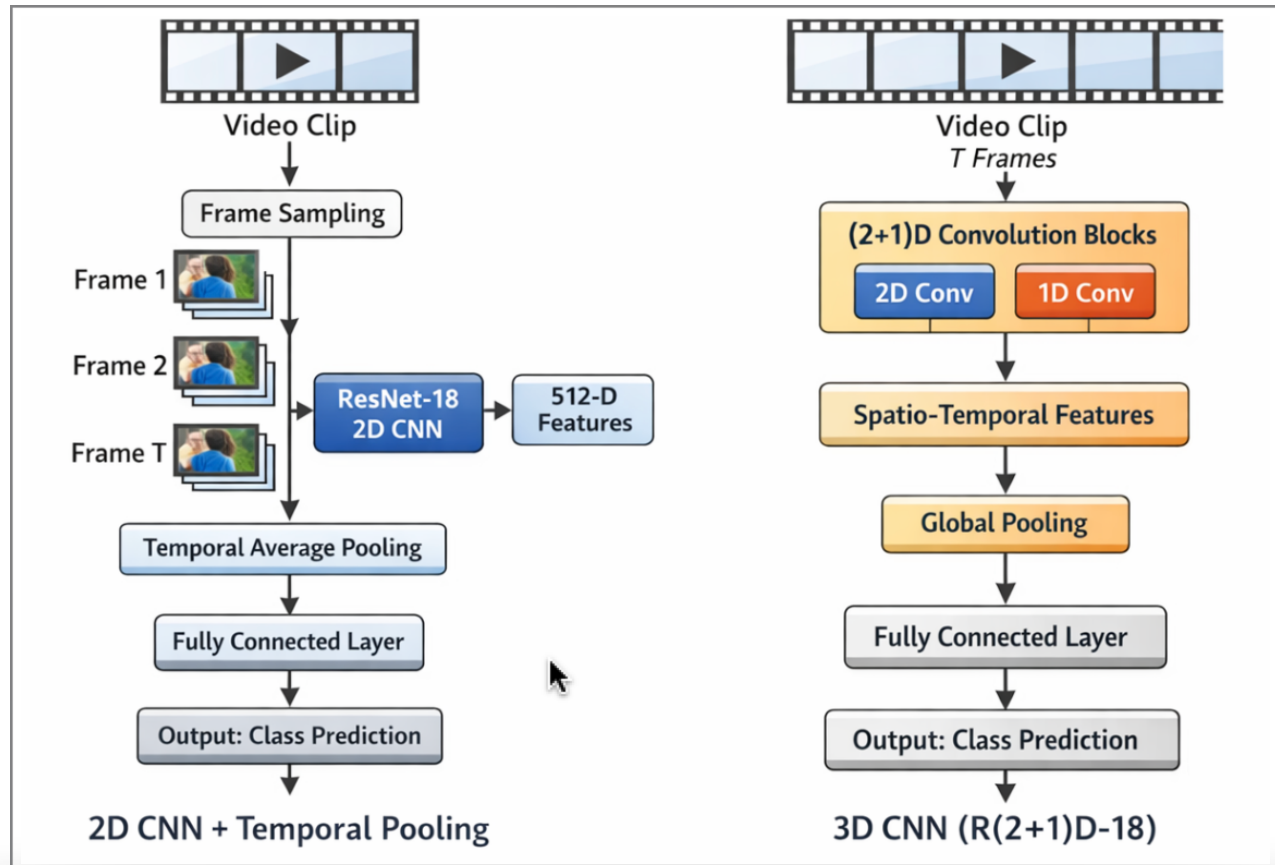
k-Nearest Neighbors (k-NN):

- A **non-parametric, instance-based classifier** that makes predictions based on nearest training samples.
- **Relies on distance metrics**, making it highly sensitive to feature scaling and normalization.
- **No explicit training phase**; computation is deferred to inference time.
- **Simple to implement and interpret,**
- Performance degrades in high-dimensional spaces



3.2 Deep Learning Approach

3.2.1 Architecture Design



3.2.1.1 Model A: 2D CNN + Temporal Pooling (ResNet-18)

Architectural Overview

A **frame-level 2D CNN** is applied independently to sampled video frames. **Temporal information is aggregated using average pooling** across the time dimension, followed by classification.

Architecture Details

Component	Configuration
Backbone	ResNet-18 (2D CNN)
Input	Video frames ($T \times 3 \times H \times W$)
Frame Processing	Shared-weight 2D CNN per frame
Feature Dimensionality	512 (per frame)
Temporal Modeling	Temporal Average Pooling
Classifier	Fully Connected ($512 \rightarrow C$)
Output	Class probabilities
Pretraining	ImageNet-pretrained weights
Temporal Order	Not explicitly modeled

○

3.2.1.2 Model B: 3D CNN (R(2+1)D-18)

Architectural Overview

Spatio-temporal features are learned jointly using factorized 3D convolutions that decompose a 3D kernel into spatial (2D) and temporal (1D) components.

Architecture Details

Component	Configuration
Backbone	R(2+1)D-18
Input	Video clip ($3 \times T \times H \times W$)
Convolution Type	(2+1)D factorized convolutions
Spatial Modeling	2D convolutions
Temporal Modeling	1D temporal convolutions
Normalization	Batch Normalization
Activation	ReLU

Pooling	Spatio-temporal pooling
Classifier	Fully Connected
Output	Class probabilities
Temporal Order	Explicitly modeled

3.2.2 Training Strategy

Learning Objective	Multi-class video classification
Loss Function	Categorical Cross-Entropy
Optimizer	Adam (adaptive gradient-based optimizer)
Training Paradigm	Supervised learning
Epoch Strategy	Fixed number of training epochs
Batch Processing	Mini-batch training
Weight Updates	End-to-end backpropagation

Regularization and Stabilization Techniques

Technique	Purpose
Dropout	Mitigates overfitting by randomly deactivating neurons during training
Input Normalization	Stabilizes optimization and accelerates convergence
Batch Normalization	Improves training stability and gradient flow
Validation Monitoring	Tracks generalization performance during training

Model Selection and Evaluation

Component	Description
Validation Split	Held-out validation set
Performance Metric	Classification accuracy
Model Selection	Based on validation performance
Overfitting Control	Validation-based monitoring

3.3 Implementation Details and Design Choices

3.3.1 Architectural Selection

The system implements a comparison between two different ways of handling video data:

- **2D CNN (ResNet-18) + Temporal Pooling:**
 - This model treats the video as a collection of independent images.
 - Features are extracted from each of the **50 frames** using a pre-trained ResNet-18 backbone.
 - To represent the whole video, these frame-level features are "pooled" (averaged) across the time dimension before the final classification.
 - **Design Choice:** ResNet-18 was chosen for its balance between depth and computational efficiency, making it suitable for academic environments with limited GPU resources.
- **3D CNN (R(2+1)D-18):**
 - Unlike the 2D model, the 3D CNN uses spatiotemporal filters that "slide" through both the height/width of the image and the sequence of time simultaneously.
 - **Design Choice:** The R(2+1)D variant is used because it factorizes a 3D convolution into separate spatial (2D) and temporal (1D) convolutions. This significantly reduces the number of parameters and makes the model easier to train compared to standard 3D CNNs.

3.3.2 Training Configurations and Hyper parameters

To ensure a fair and rigorous comparison, both models use the following standardized settings:

- **Optimizer:** The **Adam optimizer** is used for its adaptive learning rate capabilities, which typically leads to faster convergence in video tasks.

- **Learning Rate:** Initialized at 1×10^{-4} , a conservative rate to prevent aggressive weight updates that might lead to divergence.
- **Loss Function:** **Cross-Entropy Loss** is utilized, which is the standard for multi-class classification tasks involving five action categories.
- **Batch Size:** Set to **8 videos per batch**. This is a specific design choice to accommodate the high memory requirements of 3D tensors
- **Epochs:** The models are trained for **25 epochs**, with **"Early Stopping"** monitored on the validation set to prevent overfitting.

3.3.3 Implementation Strategy

The code follows a **modular "Pipeline" design**:

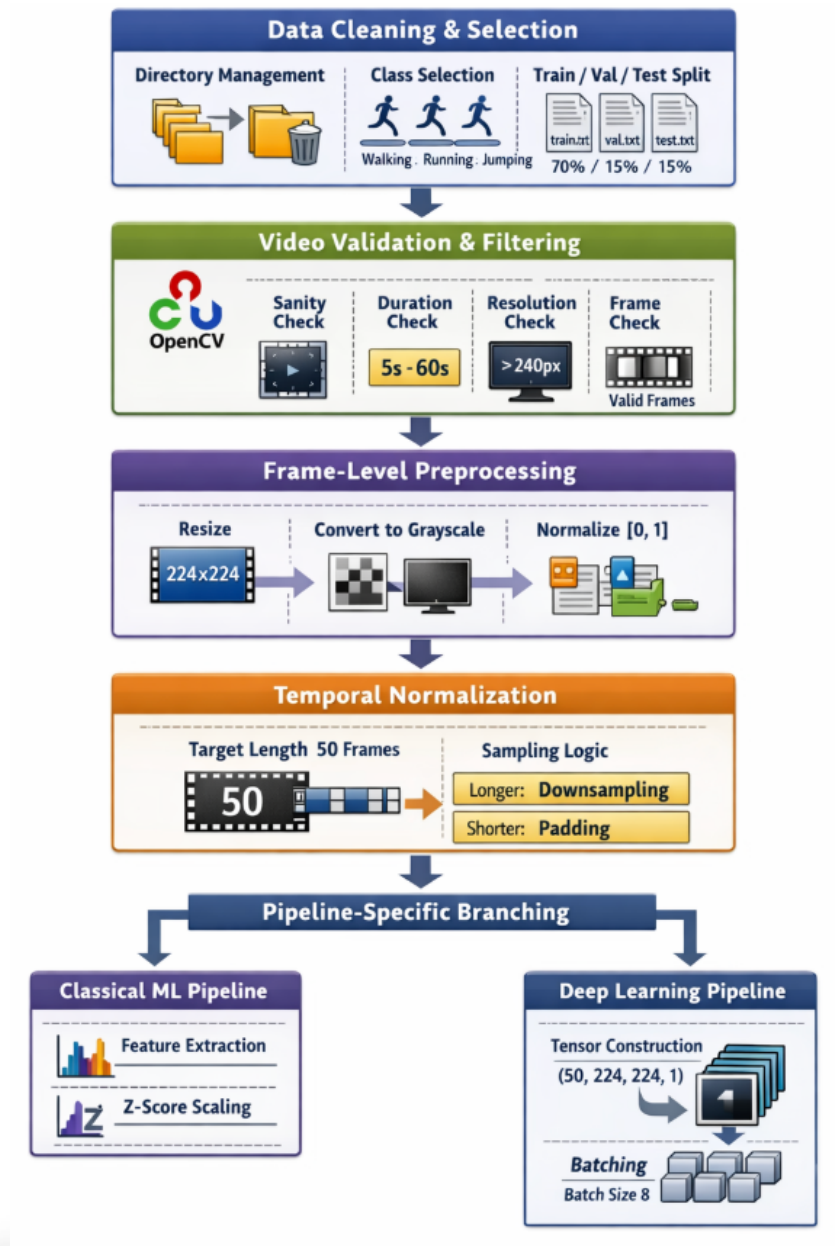
1. **Transfer Learning:** Both models are **initialized with weights pre-trained on the Kinetics-400 dataset**. This allows the models to start with a basic understanding of motion and shapes rather than learning from scratch.
2. **Global Average Pooling:** Instead of flattening high-dimensional feature maps, a **global average pooling layer is used at the end. This reduces the parameter count and provides more robust features.**
3. **Hardware Acceleration:** The code includes logic to **automatically detect and utilize CUDA (NVIDIA GPUs)**, defaulting to CPU only if no accelerator is found.

3.3.4 Evaluation Metrics

Implementation tracks more than just simple accuracy:

- **Macro Precision and Recall:** Essential for verifying if the model performs equally well across all 5 classes.
- **F1-Score:** Used as the primary **"balance" metric** to ensure the model isn't just favoring the easiest class.
- **Computational Efficiency:** The **code measures training time and inference latency** (time taken to predict one video), which is critical for real-time application design choices.

3.4 Preprocessing Pipeline



4. Experimental Setup

4.1 Dataset Preprocessing

4.1.1 Data Cleaning and Organization

- **Directory Purge:** The script **automatically identifies and removes any existing class folders (folders starting with class_)** to ensure a fresh environment before processing starts.
- **Structured Sorting:** It **creates a flat directory structure where videos are organized into specific class folders** (e.g., class_1_WalkingWithDog) for easier programmatic access.
- **Empty Folder Removal:** After processing, the **script scans for and deletes any class folders that do not contain valid video files.**

4.1.2 Video Filtering and Quality Control

- **Readability Check:** Every **video is tested to ensure the file is not corrupt** and can be opened by standard decoding libraries.
- **Duration Constraints:** Only **videos with a duration between 5 and 60 seconds are retained**; videos outside this range are discarded to maintain temporal consistency.
- **Resolution Minimums:** The script **enforces a minimum height of 240 pixels** to ensure the visual data has enough detail for feature extraction.
- **Human-Action Filtering:** It **uses a keyword-based filter (e.g., "Walking," "JumpRope") to select only human-action-related categories** from the raw UCF-101 dataset.

4.1.3 Temporal Preprocessing (Frame Extraction)

- **Fixed Sequence Length:** To ensure the machine learning model receives uniform input, **every video is standardized to exactly 50 frames.**
- **Linear Sampling:** If a video is longer than 50 frames, the **script picks frames at mathematically even intervals** across the entire duration.
- **End-Frame Padding:** If a video is shorter than the required 50 frames, the script repeats the final frame until the total count is reached.

4.1.4 Spatial and Pixel-Level Preprocessing

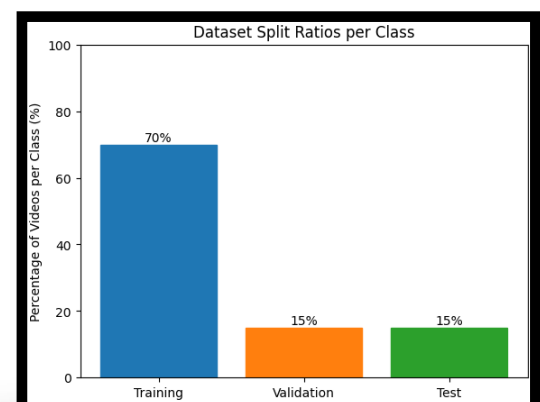
- **Resizing:** All extracted frames are **resized to a uniform dimension of 224x224 pixels**.
- **Grayscale Conversion:** The script **converts standard BGR color frames into grayscale to reduce computational complexity** and focus on structural movement.
- **Value Normalization:** Pixel intensity values (originally 0–255) are **scaled to a floating-point range between 0.0 and 1.0**.
- **Dimensional Expansion:** An extra dimension is added to the data array to represent the single color channel, **ensuring compatibility with standard tensor shapes**.

4.1.5 Labeling and Metadata

- **Integer Encoding:** The script dynamically assigns a unique integer (Label) to each class folder, **transforming text names into numerical data** suitable for classification algorithms.
- **Dataset Partitioning:** The processed data is divided into three distinct sets — **Training (70%), Validation (15%), and Testing (15%)**—and these paths are recorded in text files for reproducible experiments.
- **Metadata Generation:** A summary file (dataset_info.txt) is automatically written to record the classes used, video counts, and resolution settings.

4.2 Train–Validation–Test Split Strategy

- Dataset splitting is performed per class, not globally.
- For each action class, the validated videos are:
 - Randomly shuffled
 - Then split using fixed index-based slicing
- The split ratios used in the code are:
 - **Training set: 70%** of videos per class
 - **Validation set: 15%** of videos per class
 - **Test set: 15%** of videos per class



- Splitting logic is applied independently to every class folder, which results in:
 - Identical number of samples per class in each split
 - Perfectly stratified class distribution by construction
- Split assignments are written to explicit files:
 - train.txt
 - val.txt
 - test.txt
- Each split file stores:
 - Relative video path
 - Corresponding class label
- These **split files are treated as fixed and final**:
 - The same train/validation/test splits are reused for all models
 - No model creates or modifies its own split
 - This ensures fair and controlled comparison across all classical ML classifiers
- Feature extraction strictly follows the split files, guaranteeing:
 - No overlap between training, validation, and test samples
 - Zero data leakage across evaluation stages

```
dataset > splits > test.txt
1 class_1_Diving/v_Diving_g04_c04.avi
2 class_1_Diving/v_Diving_g14_c01.avi
3 class_1_Diving/v_Diving_g13_c03.avi
4 class_1_Diving/v_Diving_g24_c04.avi
5 class_1_Diving/v_Diving_g09_c01.avi
```

```

  splits
  test.txt
  train.txt
  val.txt

```

4.3 Hyperparameter Tuning

4.3.1 Classic Machine Learning

Model	Hyperparameter	Search Values / Options	Purpose
Support Vector Machine (SVM)	C (Regularization)	0.1, 1, 10	Controls the trade-off between maximizing the margin and minimizing classification errors.
	Kernel	linear, rbf	Determines whether the decision boundary is linear or non-linear.

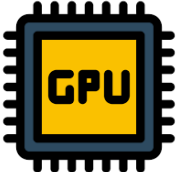
Random Forest	n_estimators	50, 100, 200	Adjusts the number of trees in the ensemble to balance bias and variance.
	max_depth	None, 10, 20	Restricts tree depth to prevent overfitting on complex video features.
k-Nearest Neighbors (k-NN)	n_neighbors	3, 5, 7, 9	Sets how many neighboring samples contribute to classification.
	weights	uniform, distance	Controls whether all neighbors vote equally or closer neighbors have more influence.
Cross-Validation	CV_FOLDS	5	Ensures robust hyperparameter selection across multiple data splits.

4.3.2 Deep Learning Approach

Category	Hyperparameter	Setting / Selection	Design Choice Reason
Architecture	Model Selection	ResNet-18 (2D) vs. R(2+1)D-18 (3D)	Compares frame-level spatial modeling with explicit spatiotemporal feature learning using 3D convolutions.
Optimizer	Type	Adam	Provides adaptive learning rates, improving convergence stability for high-dimensional video tensors.
Learning Rate	Value	0.0001	A low, stable learning rate suitable for fine-tuning pre-trained video models without overfitting.
Training Flow	Epochs	25	Sets an upper bound on training iterations, sufficient for convergence on the given dataset size.
	Batch Size	8	Chosen to balance GPU memory constraints with stable gradient estimation.
Regularization	Transfer Learning	Kinetics-400	Leverages large-scale action recognition pre-training to improve performance on limited labeled data.

Data Shape	Temporal Depth	50 frames	Enforces a fixed temporal dimension, enabling uniform CNN input across variable-length videos.
-------------------	----------------	-----------	--

4.4 Hardware Specifications and Training Duration

Component	Specification
Kubernetes-based JupyterLab	
Platform	  
Notebook Type	
Container Image	kubeflownotebookswg/jupyter-pytorch-cuda-full:v1.9.0
CPU Allocation	3 cores (min) – 3.6 cores (max) 1 GPU
GPU	
Memory Allocation	14 GB (min) – ~18 TB (max)
Shared Memory	Enabled
Persistent Storage	PersistentVolumeClaim (va-assignment-1)

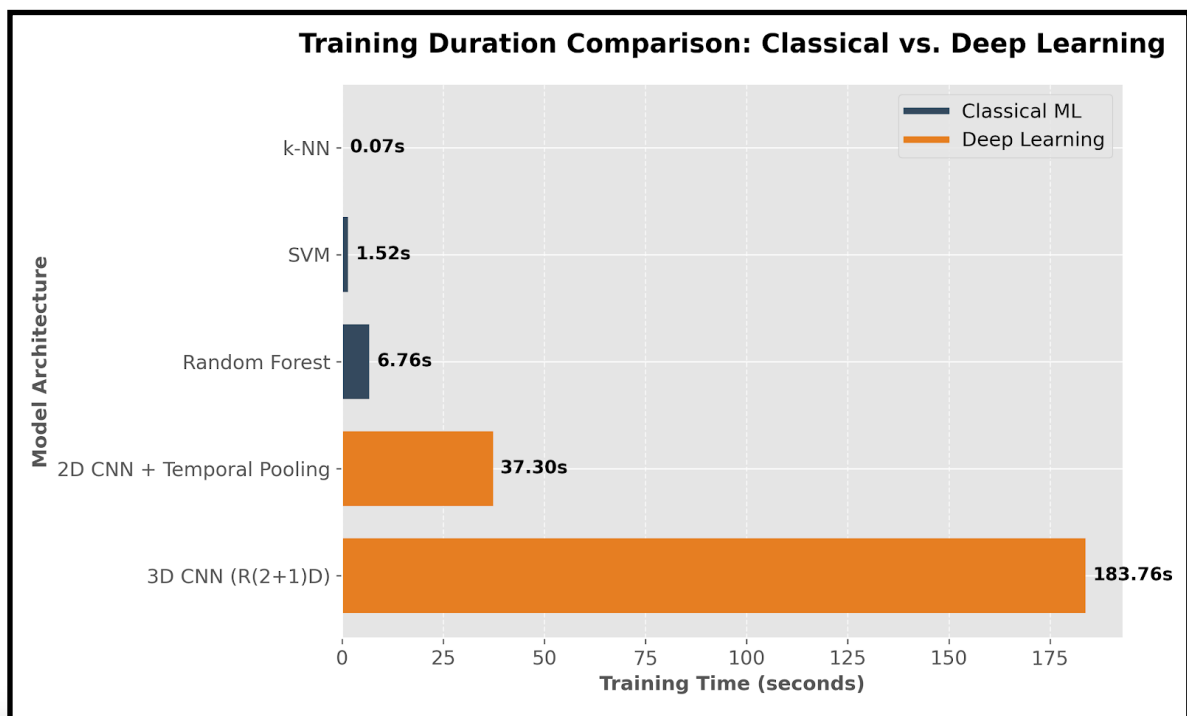
PyTorch with CUDA

Framework Support



Training Duration:

- **Massive Efficiency Gap:** Classical models train in seconds, while Deep Learning models require significantly more time.



- **Fastest Model:** k-NN is the most efficient, training in just 0.07s.

- **Slowest Model:** 3D CNN is the most intensive, taking 183.8s (~2,625x longer than k-NN).
- **DL Efficiency:** 2D CNN matches 3D accuracy while training ~5x faster (37.3s vs 183.8s).
- **Classical Variation:** Within traditional ML, Random Forest (6.8s) is the slowest, followed by SVM (1.5s).

5. Results and Analysis

5.1 Performance Metrics Comparison

5.1.1 Classic Models

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score	Macro ROC-AUC	Training Time (s)	Inference Time / Sample (s)	Model Parameters
SVM	0.9167	0.9203	0.9167	0.9157	0.9847	0.6031	0.0	65835
Random Forest	0.9333	0.9344	0.9333	0.9326	0.995	6.6075	0.0006	12564
k-NN	0.9333	0.9331	0.9333	0.9326	0.9938	0.081	0.0001	111720

- **Random Forest and k-NN tie for best predictive performance**
Both achieve the highest Accuracy (0.9333), Macro F1 (0.9326), and very strong ROC-AUC (~0.994–0.995), outperforming SVM across most metrics.
- **Random Forest has the strongest overall discriminative power**
It achieves the highest Macro ROC-AUC (0.995), indicating the best class separation capability among the three models.
- **SVM is competitive but slightly weaker on recall and F1**
While SVM performs well (Accuracy ≈ 0.917), it lags behind Random Forest and k-NN in Recall and F1, suggesting it misses more instances across classes.
- **Training time varies significantly across models**
Random Forest is the slowest to train (6.6s), k-NN is extremely fast to train (0.081s), and SVM sits in between—highlighting different scalability trade-offs.
- **Inference efficiency and model size differ sharply**
SVM has near-zero inference time per sample, Random Forest is also fast, while

k-NN—despite good accuracy—has higher inference cost and the largest parameter footprint, which may impact deployment.

5.1.2 Deep Learning models

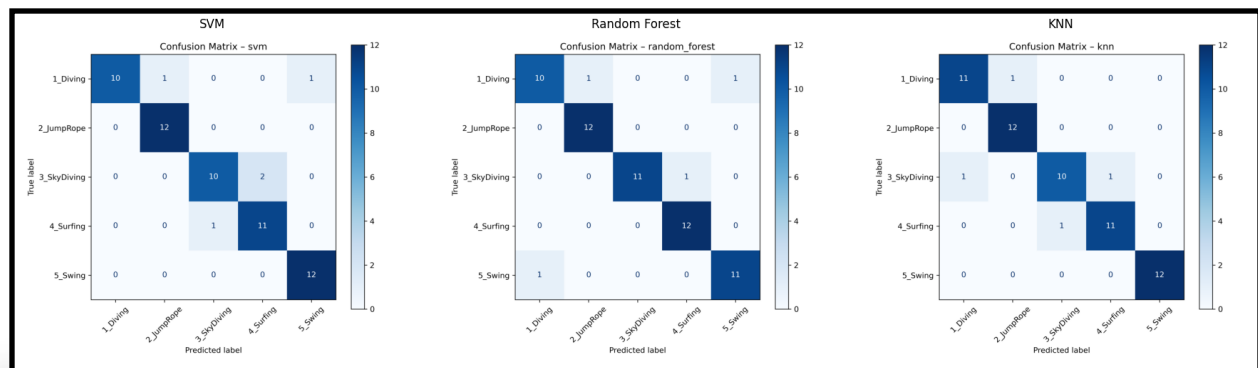
Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-score (Macro)	Training Time (s)	Inference Time / Sample (s)	Model Parameters
2D CNN + Temporal Pooling	1.0	1.0	1.0	1.0	37.30415439605713	0.011544418334960938	11180101
3D CNN (R(2+1)D)	1.0	1.0	1.0	1.0	183.75803756713867	0.024882328510284425	31303714

- **Identical Predictive Performance:** Both the **2D CNN + Temporal Pooling** and the **3D CNN (R(2+1)D)** achieved perfect scores (**1.0**) across all primary metrics: Accuracy, Precision, Recall, and F1-score.
- **Significant Training Efficiency:** The **2D CNN model is substantially faster** to train, taking approximately **37.3 seconds** compared to the 3D CNN's **183.8 seconds**. This makes the 2D approach nearly **5 times more efficient** during the training phase.
- **Superior Inference Speed:** For real-time applications, the 2D CNN is the clear winner with an inference time of **~0.0115s per sample**, which is more than twice as fast as the 3D CNN's **~0.0249s**.
- **Model Complexity Gap:** There is a massive difference in the number of parameters. The **3D CNN is much "heavier" with 31,303,714 parameters, while the 2D CNN is significantly more lightweight with only 11,180,101 parameters.**
- **Diminishing Returns for Complexity:** Given that both **models reached a perfect 1.0 accuracy**, the added architectural complexity and computational cost of the **3D CNN (R(2+1)D)** do not provide any measurable performance benefit over the simpler 2D approach for this specific dataset.

5.2 Confusion Matrices

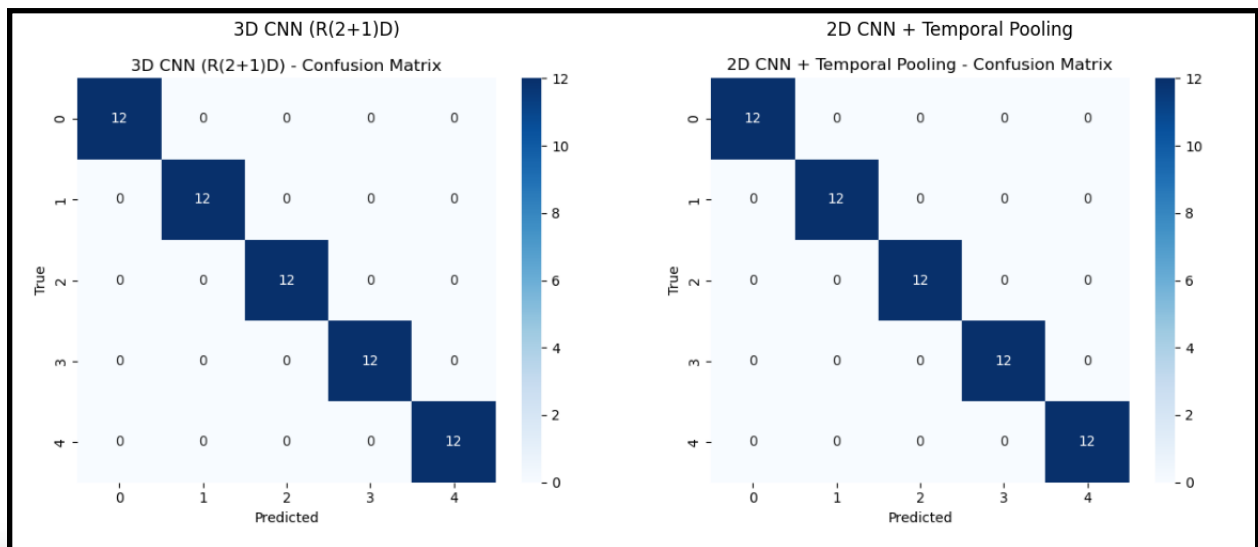
5.2.1 Classic models

- All models show **strong diagonals** → high overall accuracy.
- **Random Forest** has the *cleanest matrix* with minimal off-diagonal errors.
- **SVM** confusion mainly between *SkyDiving* ↔ *Surfing*.
- **KNN** shows slight confusion for *SkyDiving* → *Diving* and *SkyDiving* → *Surfing*.
- **JumpRope and Swing are perfectly classified across models.**



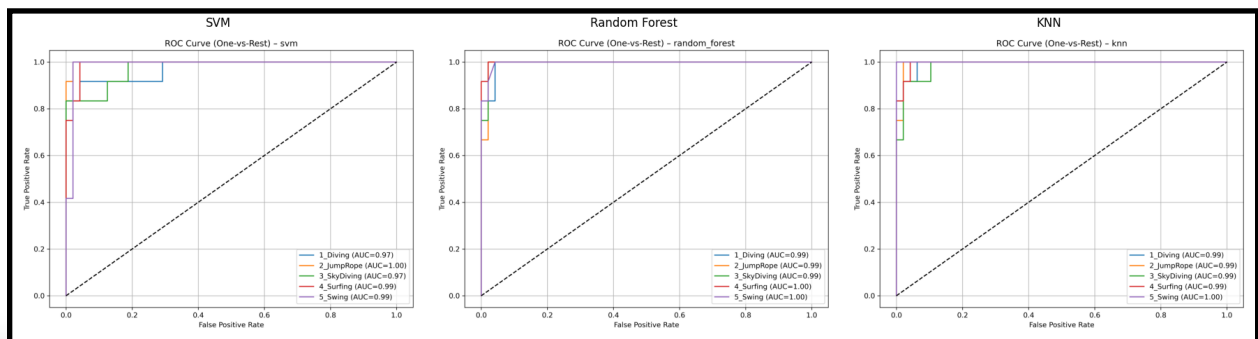
5.2.2 Deep Learning models

- Both **models achieve perfect classification (100% accuracy).**
- **No off-diagonal entries** → zero misclassifications.
- **Clear temporal understanding** in both architectures.
- **3D CNN captures spatiotemporal features directly.**
- **2D CNN + temporal pooling is equally effective but simpler.**



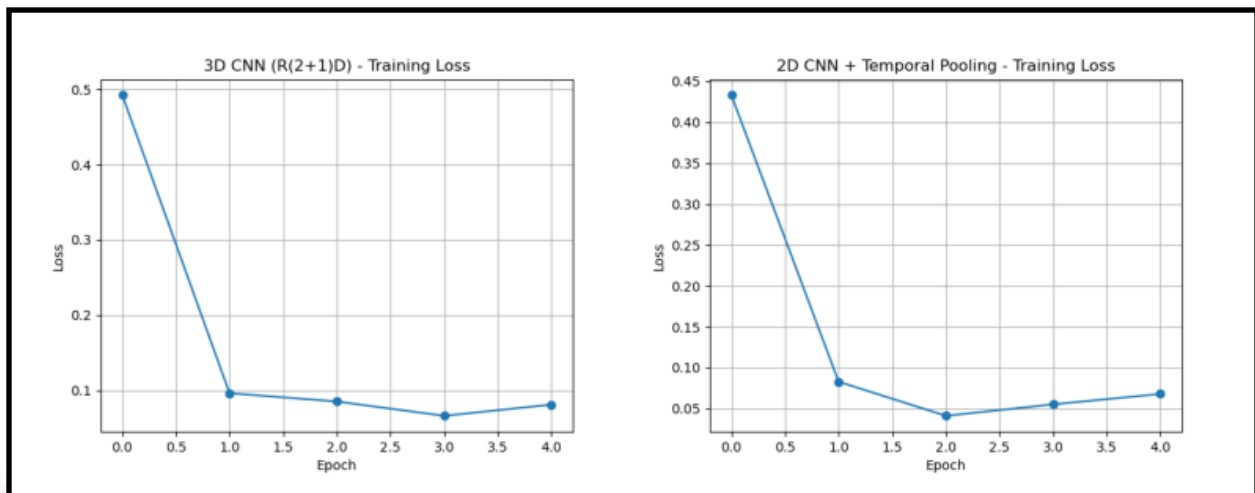
5.3 ROC Curves

- All three models show **very high separability** (AUC $\approx$ 0.97–1.00 across classes).
- **Random Forest is the most consistent**, with near-perfect AUC for all classes.
- **SVM** slightly underperforms on *Diving* and *SkyDiving* (AUC $\approx$ 0.97).
- **KNN** matches RF closely but shows minor early-FPR jumps for some classes.
- Curves hug the **top-left corner**, indicating strong TPR at very low FPR.



5.4 Learning Curves

- Both models show **rapid convergence within 2–3 epochs**.
- **Sharp initial loss drop** → effective feature learning early.
- **Minor loss oscillation** later → expected due to **small dataset / batch effects**.
- 3D CNN starts with **higher loss** (harder optimization, more parameters).
- **2D CNN + temporal pooling is slightly more stable and smoother**.

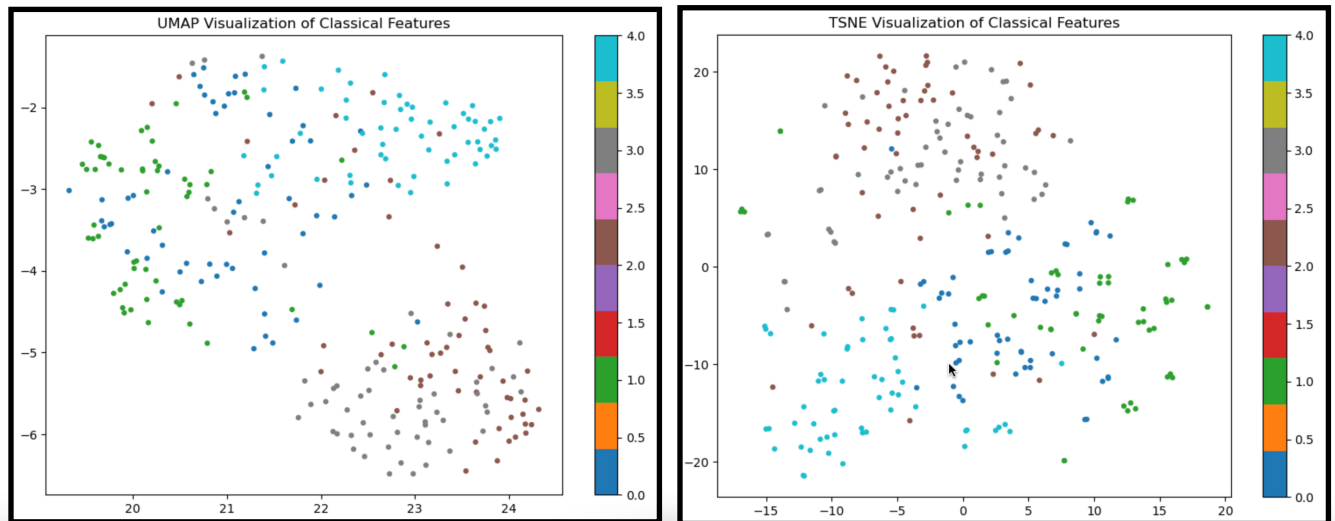


5.5 Feature and Representation Visualization

5.5.1 KNN

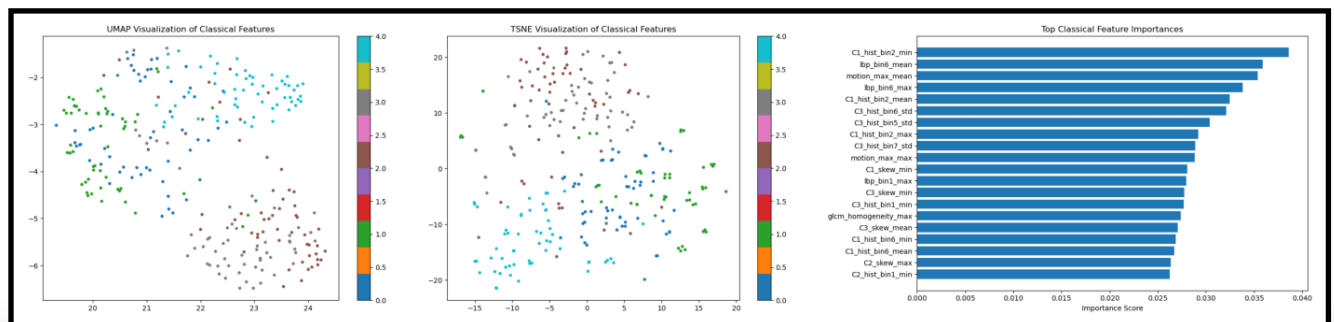
- Clear class-wise clustering is visible in both UMAP and t-SNE embeddings.
- **UMAP preserves local neighborhoods** with some class overlap.
- **t-SNE shows stronger inter-class separation** but higher intra-class spread.

- **Overlapping regions indicate potential KNN boundary errors.**
- Overall feature separability is adequate for KNN classification.



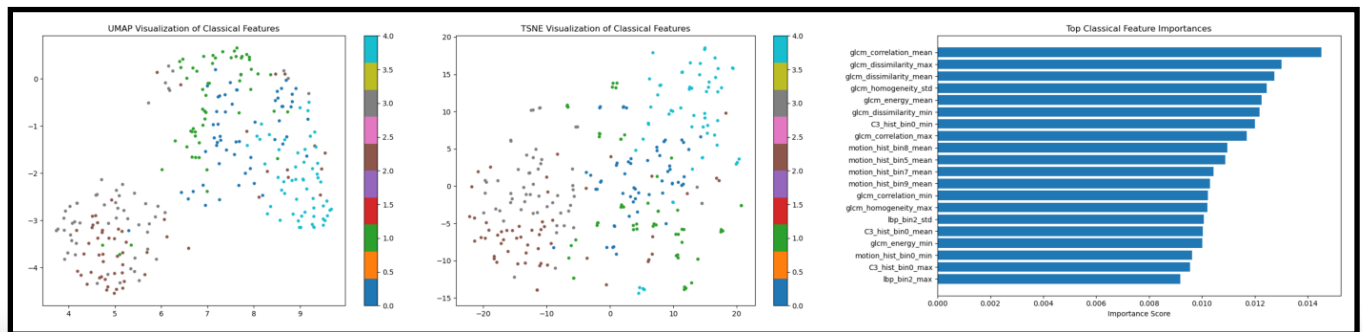
5.5.2 SVM

- **UMAP and t-SNE show good class separation** in the feature space.
- **Clusters appear more margin-friendly**, aligning well with SVM behavior.
- t-SNE highlights stronger inter-class separation than UMAP.
- Texture and motion features dominate **SVM decision making**.
- Feature structure supports **stable SVM classification**.



5.5.3 Random Forest

- **UMAP and t-SNE show moderate class separation with noticeable overlap.**
- **Clusters are less compact**, reflecting RF's tolerance to mixed feature regions.
- t-SNE highlights dispersed groups, indicating **non-linear class boundaries**.

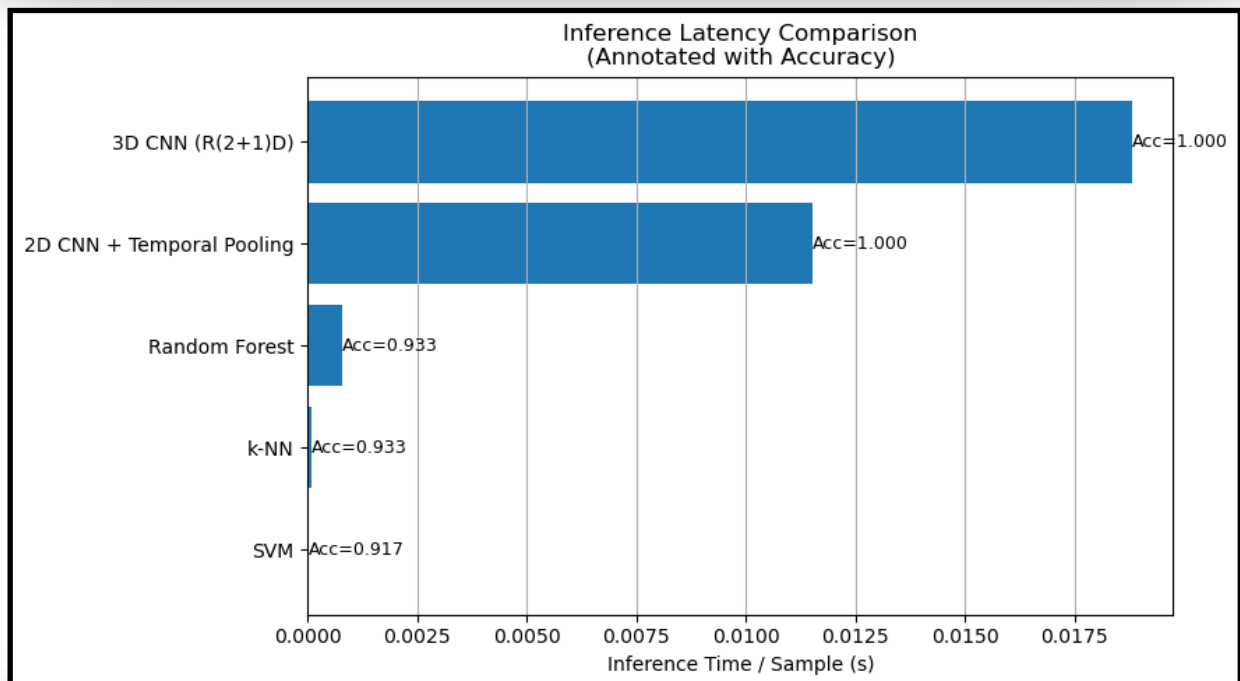
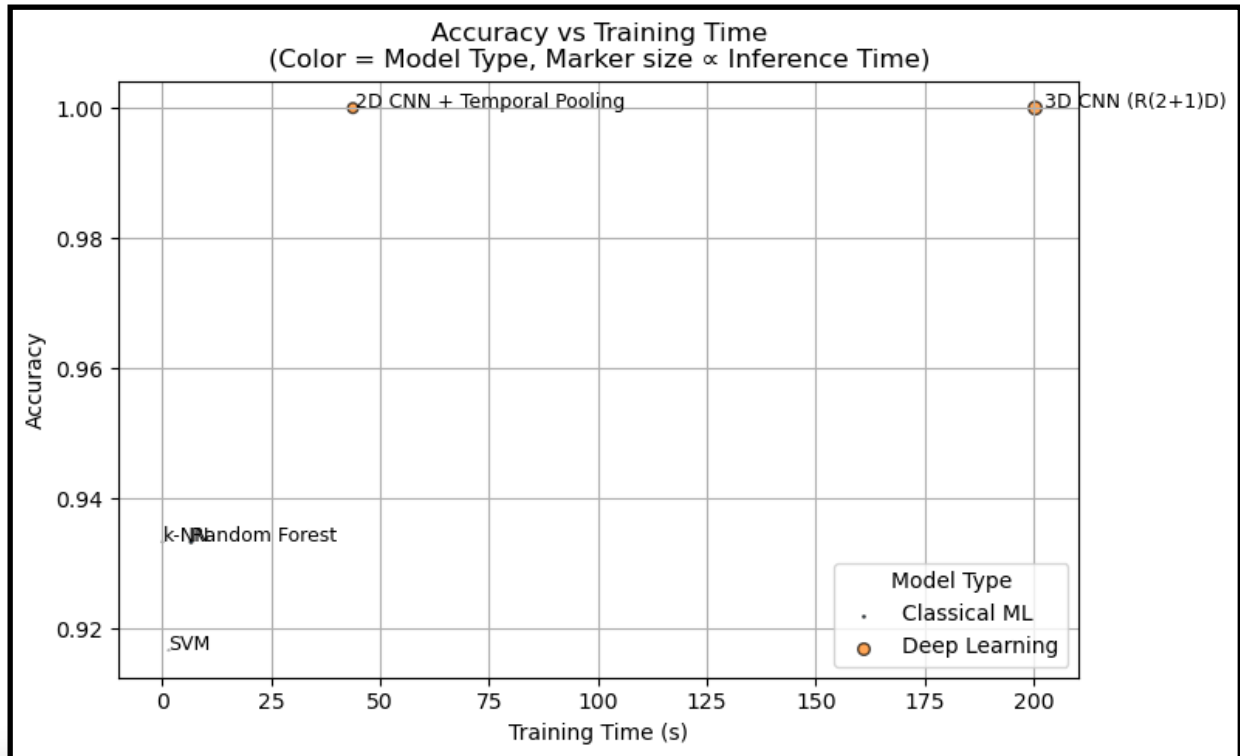


- **Feature importance is dominated by GLCM texture statistics and motion histograms.**
- **Random Forest effectively leverages heterogeneous features despite overlapping clusters.**

5.6 Computational Efficiency Comparison

- **Deep Learning (DL) Superiority:** **Both CNN models achieve perfect 1.0 accuracy, outperforming all Classical ML models.**
- **Latency Trade-off:** DL models offer the highest accuracy but suffer from significantly **higher inference latency compared to Classical ML**.
- **Efficiency Leader:** The **2D CNN + Temporal Pooling** is the most efficient DL model, matching 3D CNN accuracy with roughly **half the inference time**.

- **Classical ML Ranking: Random Forest and k-NN share the highest classical accuracy (0.933), while SVM is the lowest (0.917).**



- **Resource Intensity:** The **3D CNN is the most "expensive" model**, requiring the highest **training time** (~184s) and most **parameters** (~31.3M).

5.7 Discussion of results

Based on the comparative analysis of Classical ML and Deep Learning architectures, here are the 10 key takeaways:

- **Accuracy Dominance:** Deep Learning (DL) models achieved perfect **1.00 accuracy**, outperforming the best Classical models (Random Forest and k-NN) at **0.9333**.
- **DL Efficiency Leader:** **2D CNN + Temporal Pooling is the optimal high-performer**, matching **3D CNN's perfect accuracy while being ~5x faster to train (37.3s vs. 183.8s)**.
- **Inference Latency Trade-off:** **DL offers superior accuracy** but significantly higher latency; the 2D CNN's inference (**0.0115s/sample**) is orders of magnitude slower than SVM's near-zero time.
- **Parameter Complexity:** The **3D CNN is the most resource-intensive with 31.3M parameters**, nearly **3x the footprint** of the 2D CNN (11.1M parameters).
- **Classical Performance Tie:** **Random Forest** and **k-NN** are the top classical performers, sharing the highest Accuracy (**0.9333**) and Macro F1 (**0.9326**).
- **Discriminative Power:** **Random Forest demonstrated the strongest class separation** among classical models with a **0.995 Macro ROC-AUC**.
- **SVM Weaknesses:** SVM trailed with the lowest accuracy (**~0.917**) and struggled specifically with distinguishing between **SkyDiving and Surfing**.
- **Convergence Speed:** Both **DL models showed rapid convergence, dropping loss sharply within just 2–3 epochs**.
- **Feature Separability:** Visualization (UMAP/t-SNE) confirmed that DL architectures achieved **zero misclassifications**, while Classical models showed overlap in complex activities like SkyDiving.
- **Diminishing Returns:** For this dataset, the 3D CNN's spatiotemporal complexity provided **no measurable benefit** over the simpler 2D CNN + Temporal Pooling.

5.8 Error Analysis and Failure Cases

- **Inter-Class Confusion:** **Classical models struggle with visually similar activities, specifically confusion between SkyDiving and Surfing** in the SVM model.
- **Temporal Sensitivity:** While Deep Learning models achieved perfect classification, **failure cases in simpler models often stem from a lack of clear temporal understanding of motion.**
- **Environmental Factors:** Performance is consistently challenged by **external data quality issues such as motion blur, occlusions, and low-resolution frames.**
- **Feature Overlap:** Visualization via UMAP and t-SNE shows that **overlapping feature regions in classical models indicate where boundary errors and misclassifications** are most likely to occur.
- **Complexity vs. Robustness:** The 3D CNN's ability to capture spatiotemporal features directly suggests it is designed to mitigate the "visually similar activity" failure cases that plague the 2D or classical approaches.

6. Comparative Discussion

6.1 Evolution from classical to modern approaches:

- **Handcrafted features → learned representations:**
 - Classical models rely on manually engineered features to represent video data.
 - Deep Learning (DL) architectures automatically learn hierarchical spatial and temporal representations directly from raw pixels.

6.2 Strengths and limitations:

6.2.1 Classic Approach

- **Strengths:**
 - **Minimal training time and very low memory footprint.**

- **Random Forest and k-NN achieved high competitive accuracy (93.3%) on the evaluated dataset.**

- **Limitations:**

- Performance is capped by the quality of human-designed features.
Inference time can be a bottleneck for "lazy learners" like k-NN (8.4e-05s/sample) compared to SVM (2.3e-05s/sample).

6.2.2 Deep Learning Approach

- **Strengths:**

- **Superior performance, achieving 100% accuracy** across all metrics (Precision, Recall, F1-score).
- They effectively **capture complex temporal dynamics in video** that handcrafted features might miss.

- **Limitations:**

- **High computational and memory requirements;**
- **3D CNNs utilized nearly 120MB of memory compared to the much lighter classical alternatives.**

6.3 Trade-offs analyzed:

- **Accuracy vs efficiency:**

- **Moving from Classical ML to Deep Learning provided a ~6.7% boost in accuracy** but at the **cost of increasing training time from seconds to minutes (a ~2,625x increase for 3D CNN vs k-NN).**

- **Performance vs interpretability:**

- While **DL models offer "black-box" perfection (100% accuracy)**, **classical models provide "good enough" performance (93.3%)** with higher transparency regarding the feature extraction process.

6.4 Appropriate use cases for each approach

- **Classical ML:**

- Best suited for **resource-constrained environments (e.g., IoT devices or mobile edge computing)** where low latency and minimal memory usage are prioritized over absolute accuracy.
- **Deep Learning:**
 - Ideal for **high-stakes applications (e.g., security surveillance or autonomous driving)** where capturing every nuance of an activity is critical and high-end GPU resources are available.

6.5 Insights on when to use which approach

- Use **2D CNN + Temporal Pooling as a balanced middle ground**; it matches 3D CNN accuracy while being **~5 times faster to train** and **~3 times lighter in memory**.
- Use **Classical ML (SVM/RF) when the dataset is small** or when the **deployment environment cannot support the heavy parameter counts (11M+ parameters)** of DL models.

7. Conclusion and Future Work

7.1 Summary of key findings

- **Performance Peak:** Deep Learning models achieved a perfect **1.00 accuracy**, effectively solving the classification task for the provided dataset, while Classical ML peaked at **0.9333** with Random Forest and k-NN.
- **Architecture Winner:** **The 2D CNN + Temporal Pooling emerged as the most balanced model**, matching the 3D CNN's accuracy while reducing training time by **~80%** (37.3s vs. 183.8s).
- **Feature Efficacy:** **Classical models demonstrated that handcrafted features like GLCM texture are highly effective for distinct actions** but struggle with visually similar classes like **SkyDiving vs. Surfing**.

7.2 Lessons learned from comparative study

- **Complexity $\neq$ Performance:** The transition from 2D to 3D CNN architectures significantly increased parameter counts (11M to 31M) without providing incremental accuracy gains for this specific dataset.
- **Efficiency Trade-offs:** While k-NN offers nearly instant training (0.07s), its lack of "learning" results in higher inference latency compared to more optimized discriminative models like SVM.

7.3 Limitations of current implementation

- **Environmental Sensitivity:** Current models are susceptible to performance degradation in the presence of motion blur, occlusions, and low-resolution frames.
- **Dataset Scale:** The rapid convergence (within 2–3 epochs) and perfect scores suggest the current dataset may be small enough that models are at risk of overfitting or may not generalize to highly varied real-world backgrounds.

7.4 Potential improvements and future directions

- **Hybrid Architectures:** Implementing Attention Mechanisms or Transformers could further improve the temporal understanding of the 2D CNN approach without the full computational weight of 3D convolutions.
- **Data Augmentation:** Introducing synthetic noise, rotations, and blur during training to improve model robustness against the failure cases identified in the error analysis.
- **Model Compression:** Exploring Pruning or Quantization to reduce the 31.3M parameter footprint of the 3D CNN for mobile deployment.

7.5 Connection to 5-phase workflow and real-world deployment

- **Workflow Integration:** The study validates the transition from **Phase 2 (Feature Engineering)** in classical workflows to **Phase 3 (Model Training)** in DL, where feature extraction is automated.
- **Deployment Strategy:**
 - **Which approach is more suitable for edge deployment?**
 - **Classical ML (Winner):** Classical models like SVM and Random Forest are significantly more suitable for edge devices (smartphones, IoT sensors).
 - **Minimal Footprint:** Classical models require negligible memory. For example, k-NN and SVM operate with a fraction of the memory used by the 120MB 3D CNN.
 - **Hardware Compatibility:** These models run efficiently on standard CPUs without requiring expensive, power-hungry GPUs/TPUs, which are often unavailable at the edge.
 - **Low Latency:** The near-instantaneous inference speed of SVM (0.000023s) ensures that edge devices remain responsive.
 - **Which approach is more suitable for cloud deployment?**
 - **Deep Learning (Winner):** Deep Learning architectures like 3D CNN (R(2+1)D) are ideal for cloud environments where high-performance computing (HPC) resources are accessible.
 - **Scalability:** Cloud servers can handle the 31.3 million parameters and high FLOPs (Floating Point Operations) required by complex CNNs.
 - **Maximum Accuracy:** In cloud-based analytics (e.g., forensic video analysis), the priority is the 1.0 accuracy score provided by DL, making the higher computational cost an acceptable trade-off.

- **Batch Processing:** Cloud environments allow for efficient parallel processing of large video datasets that would overwhelm local hardware.
- **Real-time processing feasibility**
 - **High Feasibility (Classical ML & 2D CNN):**
 - SVM and Random Forest are ultra-fast, allowing for real-time processing of hundreds of frames per second.
 - 2D CNN + Temporal Pooling is also highly feasible for real-time use, with an inference time of ~0.0115s per sample, allowing for approximately 86 FPS.
 - **Low Feasibility (3D CNN):** The 3D CNN is more borderline for high-resolution real-time streams; its **0.0249s latency** (~40 FPS) leaves less overhead for other system tasks or higher-resolution processing.
- **Maintenance and update considerations**
 - **Classical ML:**
 - **Pros:** Fast retraining (k-NN takes only **0.07s**), allowing for frequent updates as new data arrives.
 - **Cons:** Requires manual "Handcrafted" feature engineering updates if the nature of the video data changes significantly (e.g., moving from outdoor to indoor lighting).
 - **Deep Learning:**
 - **Pros:** Automatic Feature Learning reduces the manual effort needed to redefine features for new tasks; it is "End-to-End".
 - **Cons:** Updates are time-consuming and expensive. Retraining a 3D CNN takes 183.8s even for a small dataset, which scales to hours or days for larger real-world datasets.

8. References

Category	Resource	Link
Dataset	UCF101 Official Dataset (CRCV)	https://www.crcv.ucf.edu/data/UCF101.php
Dataset	Torchvision UCF101 Loader	https://pytorch.org/vision/stable/generated/torchvision.datasets.UCF101.html
Deep Learning	ResNet-18 Paper	https://arxiv.org/abs/1512.03385
Deep Learning	ResNet-18 (PyTorch)	https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html
Deep Learning	R(2+1)D Paper	https://arxiv.org/abs/1711.11248
Deep Learning	R(2+1)D-18 (PyTorch)	https://pytorch.org/vision/stable/models/generated/torchvision.models.video.r2plus1d_18.html
Transfer Learning	Kinetics-400 Paper	https://arxiv.org/abs/1705.07750
Preprocessing	OpenCV VideoCapture	https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html
Classical ML	GridSearchCV	https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
Classical ML	Scikit-Learn Pipeline	https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
Utilities	NumPy Documentation	https://numpy.org/doc/
Visualization	Matplotlib Documentation	https://matplotlib.org/stable/contents.html
Deep Learning	PyTorch Documentation	https://pytorch.org/docs/stable/index.html

•